CPSC 375 High-Performance Computing

Lecture 11

Threads

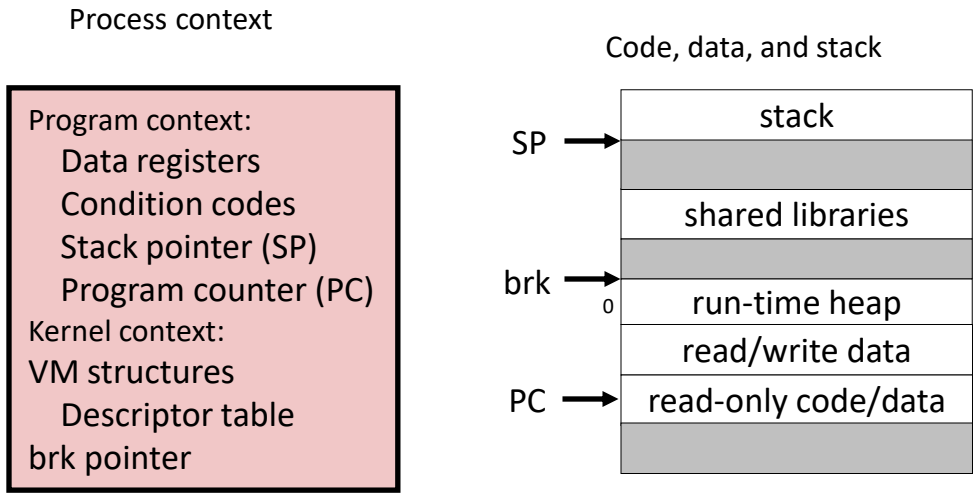
Basic Concepts

Approaches to Concurrency

- Processes
  - Hard to share resources: easy to avoid unintended sharing
  - High overhead in adding/removing clients
- Threads
  - Easy to share resources: perhaps too easy
  - Medium overhead
  - Not much control over scheduling policies
  - Difficult to debug
    - Event orderings not repeatable
- I/O multiplexing
  - Tedious and low-level
  - Total control over scheduling
  - Very low overhead
  - Cannot create as fine-grained a level of concurrency
  - Does not make use of multi-core

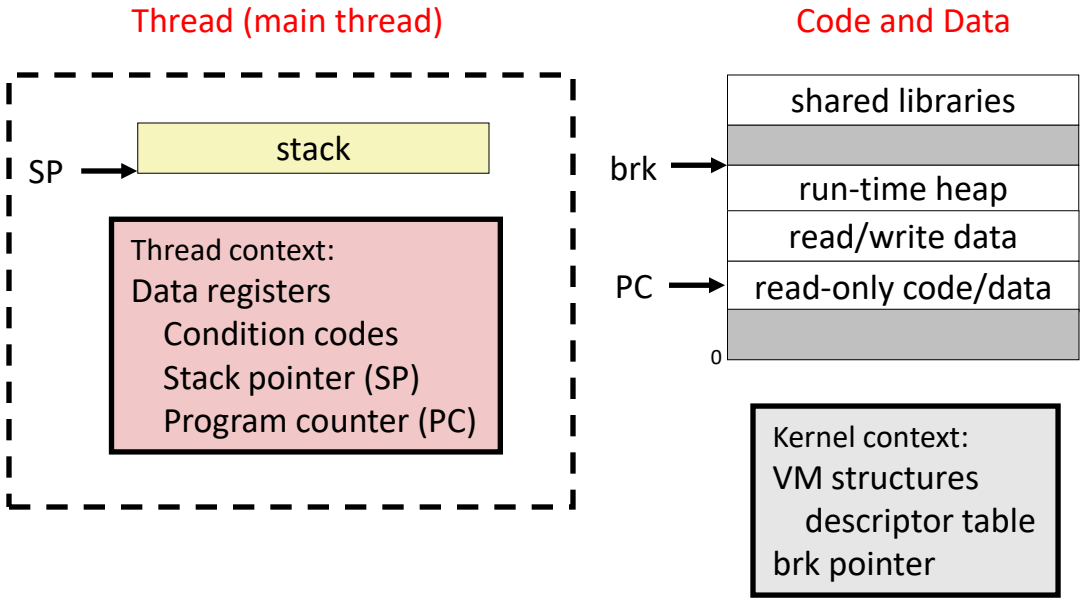
Traditional View of a Process

- Process = process context + code, data, and stack



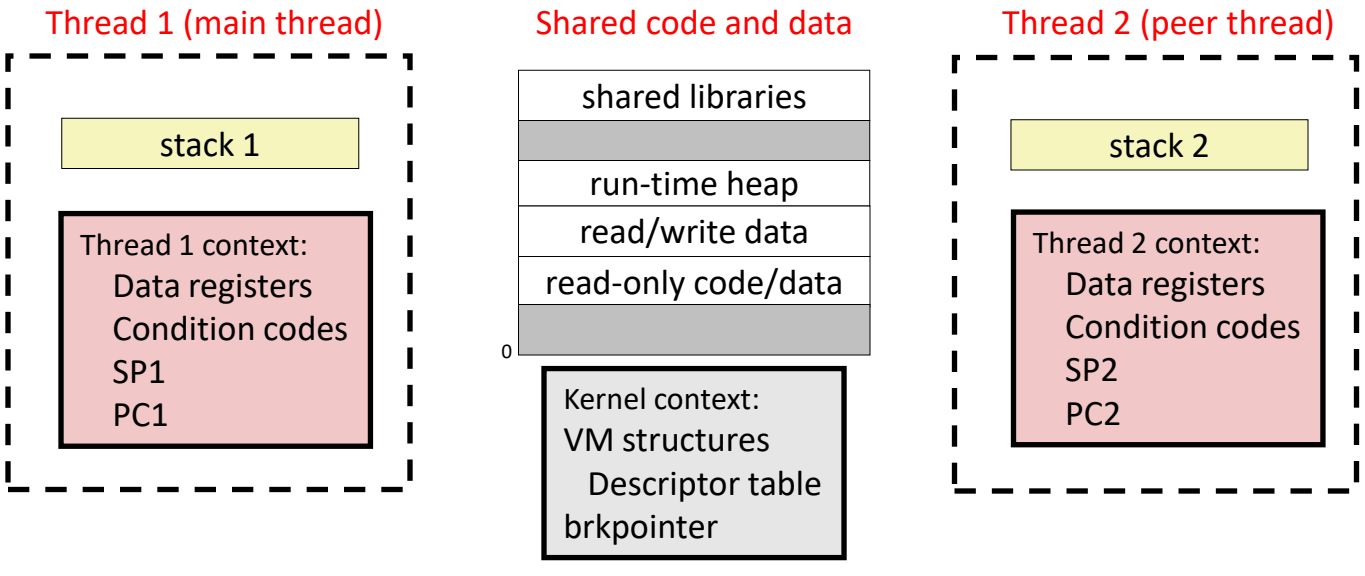
Alternate View of a Process

- Process = *thread* + code, data, and kernel context



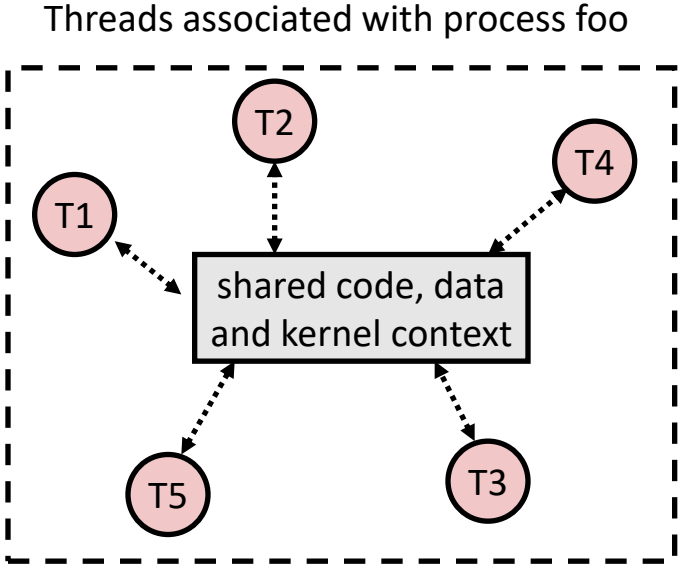
A Process with Multiple Threads

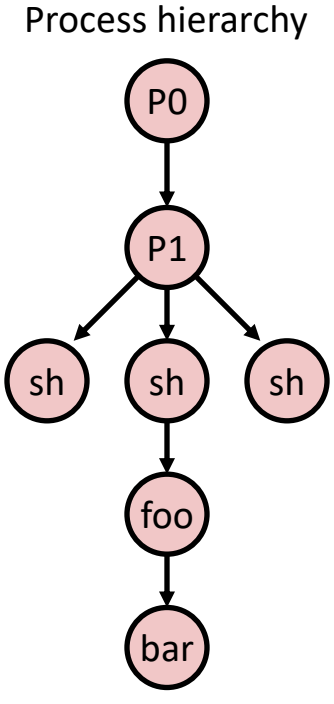
- Multiple threads can be associated with a process
  - Each thread has its own logical control flow
  - Each thread shares the same code, data, and kernel context
  - Each thread has its own thread id (TID)



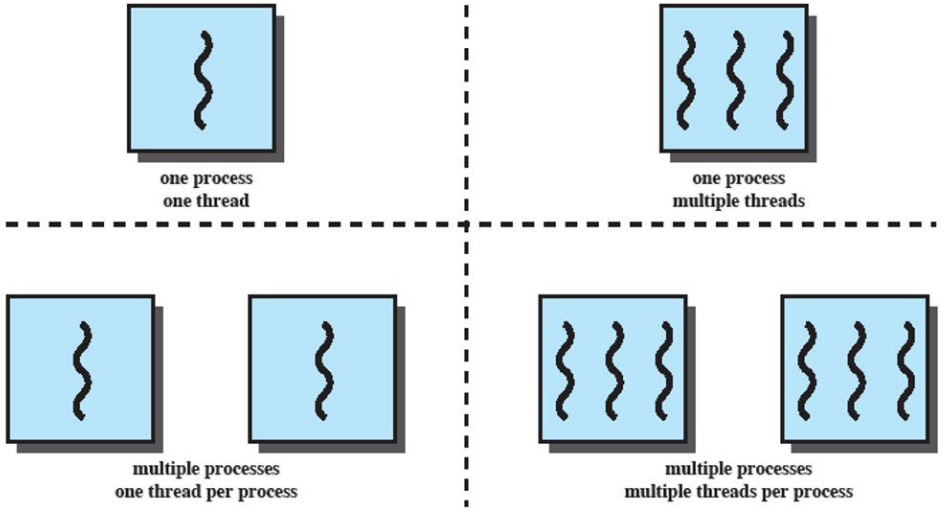
Logical View of Threads

- Threads associated with process form a pool of peers
  - Unlike processes which form a tree hierarchy





Processes and Threads



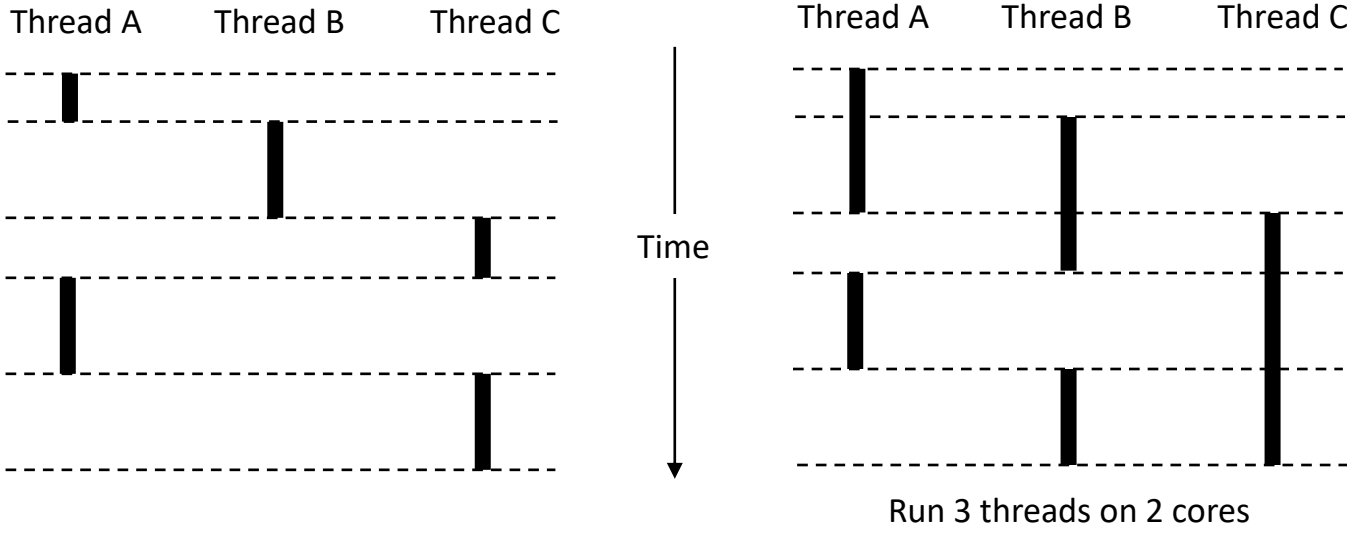
Thread Execution

Single Core Processor

- Simulate concurrency by *time slicing*

Multi-Core Processor

- Can have true concurrency



How do you write code for multi-core processors?

Threads vs. Processes

- How threads and processes are similar
  - Each has its own logical control flow
  - Each can run concurrently with others (possibly on different cores)
  - Each is context-switched
- How threads and processes are different
  - Threads share code and some data
    - Processes (typically) do not
  - Threads are somewhat less expensive than processes
    - Process control (creating and reaping) is twice as expensive as thread control
    - Linux numbers:
      - ~20K cycles to create and reap a process
      - ~10K cycles (or less) to create and reap a thread

Pros and Cons of Thread-Based Designs

- Easy to share data structures between threads
  - E.g., logging information, file cache
- Threads are more efficient than processes

But

- Unintentional sharing can introduce subtle and hard-to-reproduce errors!
  - Ease of data sharing is the greatest strength of threads
  - Also greatest weakness!

Shared Variables in Threaded C Programs

- Q: Which variables in a threaded C program are shared variables?
  - Answer not as simple as "global variables are shared" and "stack variables are private"
- Requires answers to the following questions:
  - What is the memory model for threads?
  - How are variables mapped to memory instances?
  - How many threads reference each of these instances?

Threads Memory Model

- Conceptual model:
  - Each thread runs in the context of a process
  - Each thread has its own separate thread context
    - Thread ID, stack, stack pointer, program counter, condition codes, and general purpose registers
  - All threads share remaining process context
    - Code, data, heap, and shared library segments of process virtual address space
    - Open files and installed handlers
- Operationally, this model is not strictly enforced:
  - Register values are truly separate and protected
  - But any thread can read and write the stack of any other thread

- *Mismatch between conceptual and operational model is a source of confusion and errors*

Threads Accessing Another Thread's Stack

```
char **ptr;  /* global */

int main()
{
    int i;
    pthread_t tid;
    char *msgs[N] = {
        "Hello from foo",
        "Hello from bar"
    };
    ptr = msgs;
    for (i = 0; i < 2; i++)
        Pthread_create(&tid,
            NULL,
            thread,
            (void *)i);
    Pthread_exit(NULL);
}
```

```
/* thread routine */
void *thread(void *vargp)
{
    int myid = (int)vargp;
    static int svar = 0;

    printf("[%d]: %s (svar=%d)\n",
        myid, ptr[myid], ++svar);
}
```

Peer threads access main thread's stack indirectly through global ptr variable

Mapping Vars to Memory Instances

Global var: 1 instance (`ptr` [data])

Local automatic vars: 1 instance (`i.m, msgs.m`)

```
char **ptr;  /* global */

int main()
{
    int i;
    pthread_t tid;
    char *msgs[N] = {
        "Hello from foo",
        "Hello from bar"
    };
    ptr = msgs;
    for (i = 0; i < 2; i++)
        pthread_create(&tid,
            NULL,
            thread,
            (void *)i);
    pthread_exit(NULL);
}
```

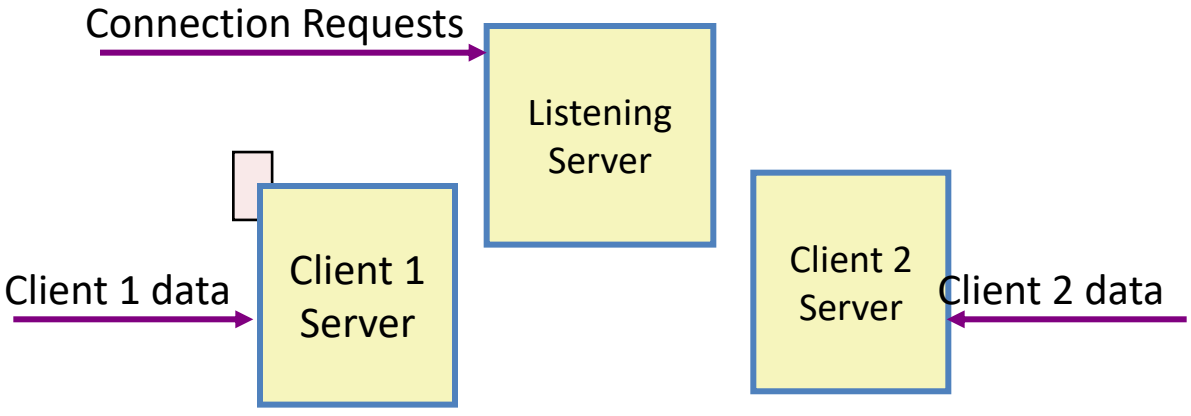
Local automatic var: 2 instances (
`myid.p0`[peer thread 0's stack],
`myid.p1`[peer thread 1's stack]
)

```
/* thread routine */
void *thread(void *vargp)
{
    int myid = (int)vargp;
    static int cnt = 0;

    printf("[%d]: %s (cnt=%d)\n",
        myid, ptr[myid], ++cnt);
}
```

Local static var: 1 instance (`cnt` [data])

Threaded Execution Model



- Multiple threads within single process
- Some state between them
  - File descriptors

Need for Synchronization

```
/* badcnt.c */
int cnt = 0; /* global */

int main(int argc, char **argv)
{
  int niters = atoi(argv[1]);
  pthread_t tid1, tid2;

  pthread_create(&tid1, NULL,
                 thread, &niters);
  pthread_create(&tid2, NULL,
                 thread, &niters);
  pthread_join(tid1, NULL);
  pthread_join(tid2, NULL);

  /* Check result */
  if (cnt != (2 * niters))
    printf("BOOM! cnt=%d\n", cnt);
  else
    printf("OK cnt=%d\n", cnt);
  exit(0);
}
```

```
/* Thread routine */
void *thread(void *vargp)
{
  int i, niters = *((int *)vargp);

  for (i = 0; i < niters; i++)
    cnt++;

  return NULL;
}
```

```
linux> ./badcnt 10000
OK cnt=20000
linux> ./badcnt 10000
BOOM! cnt=13051
linux>
```

`cnt` should equal 20,000.

What went wrong?

Shared Variables in Threaded C Programs

- *Q*: Which variables in a threaded C program are *shared*?
- A variable **x** is *shared* if multiple threads reference some instance of **x**.

Assembly Code for Counter Loop

C code for counter loop in thread i

```
for (i=0; i < niters; i++)
    cnt++;
```

Corresponding assembly code

```
        movl (%edi),%ecx
        movl $0,%edx
        cmpl %ecx,%edx
        jge .L13
.L11:
```

} Head (H_i)

```
        movl cnt,%eax
        incl %eax
        movl %eax,cnt
```

} Load `cnt` (L_i)
Update `cnt` (U_i)
Store `cnt` (S_i)

```
        incl %edx
        cmpl %ecx,%edx
        jl .L11
.L13:
```

} Tail (T_i)

What Is "Sequential Consistency?"

- Two (or more) parallel executions are sequentially consistent iff instructions of each thread (or process) are executed in sequential order
  - No restrictions on how threads relate to each other
  - Each thread runs at arbitrary speed
  - Any interleaving is legitimate
    - Any (or all) instructions of B can run between any two instructions of A

Concurrent Execution

- In general, any sequentially consistent interleaving is possible, but some give an unexpected result!
  - I_i denotes that thread i executes instruction I
  - $\%eax_i$ is the content of %eax in thread i's context

i (thread)	I_i	$\%eax_1$	$\%eax_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2

Thread 1 critical section

Thread 2 critical section

OK

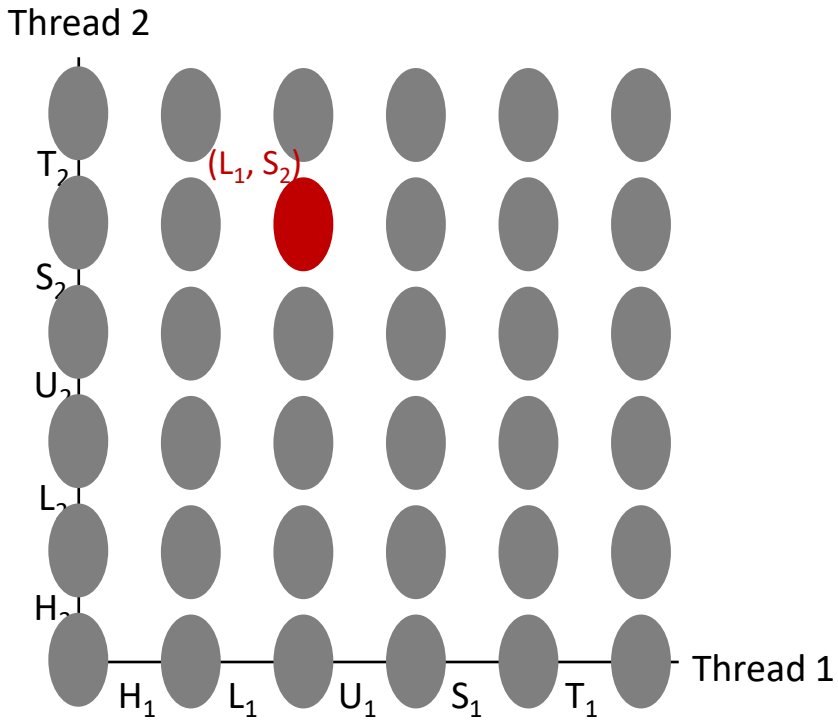
Concurrent Execution (cont)

- Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2

i (thread)	I_i	$\%eax_1$	$\%eax_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
2	H_2	-	-	0
2	L_2	-	0	0
1	S_1	1	-	1
1	T_1	1	-	1
2	U_2	-	1	1
2	S_2	-	1	1
2	T_2	-	1	1

Oops!

Progress Graphs



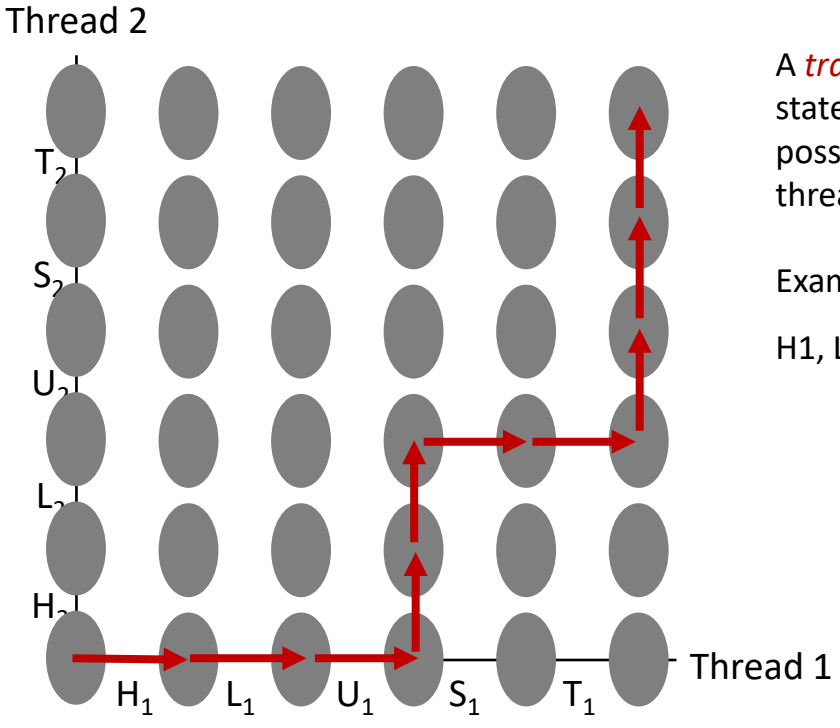
A *progress graph* depicts the discrete *execution state space* of concurrent threads.

Each axis corresponds to the sequential order of instructions in a thread.

Each point corresponds to a possible *execution state* (I_1, I_2).

E.g., (L_1, S_2) denotes state where thread 1 has completed L_1 and thread 2 has completed S_2.

Trajectories in Progress Graphs

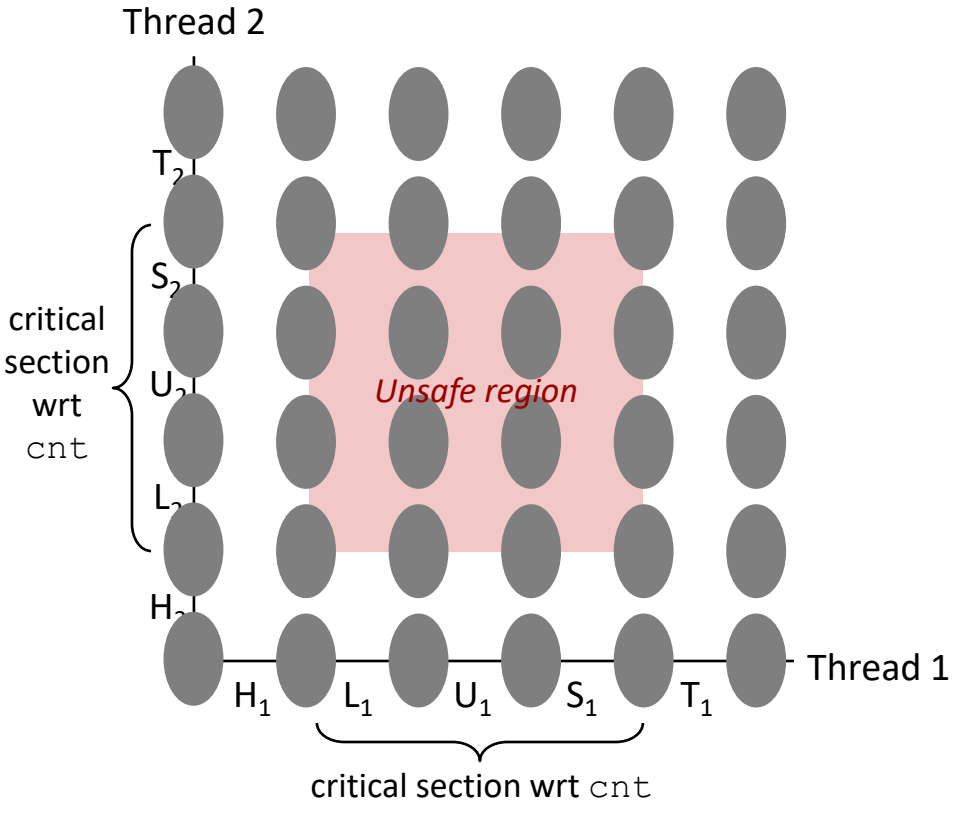


A *trajectory* is a sequence of legal state transitions that describes one possible concurrent execution of the threads.

Example:

H1, L1, U1, H2, L2, S1, T1, U2, S2, T2

Critical Sections and Unsafe Regions

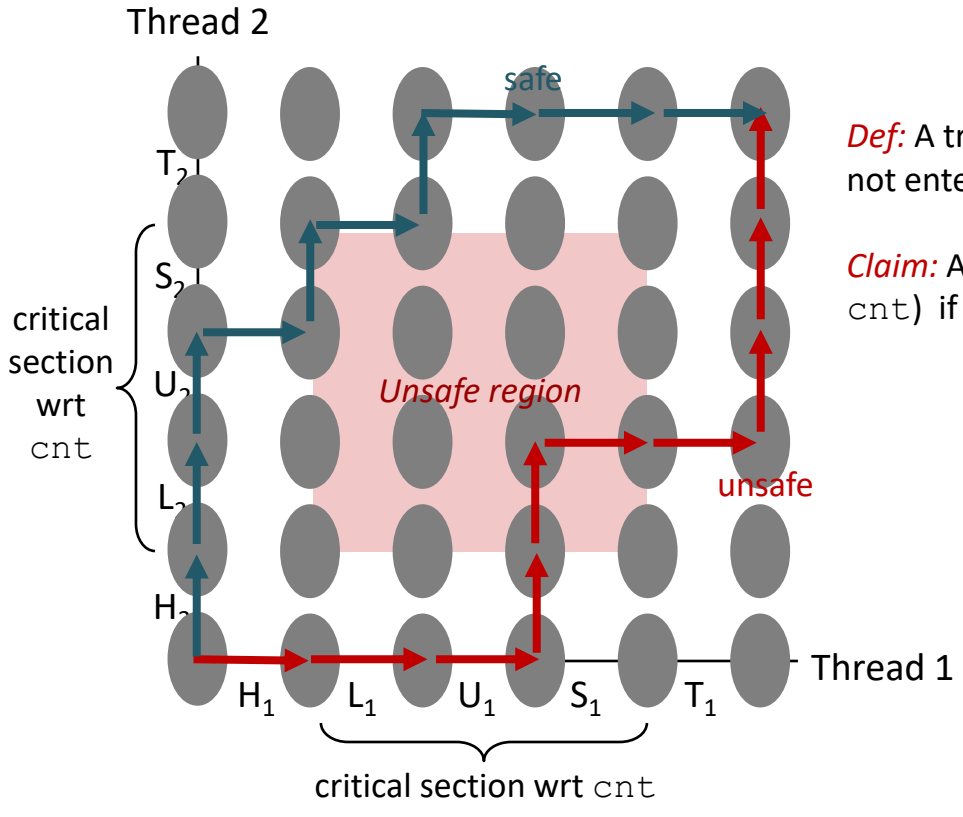


L, U, and S form a *critical section* with respect to the shared variable `cnt`

Instructions in critical sections (wrt to some shared variable) should not be interleaved

Sets of states where such interleaving occurs form *unsafe regions*

Critical Sections and Unsafe Regions



Def: A trajectory is *safe* if it does not enter any unsafe region.

Claim: A trajectory is correct (wrt `cnt`) if it is safe.

Enforcing Mutual Exclusion

- *Q:* How can we guarantee a safe execution of the threads?
- *A:* We must *synchronize* the execution of the threads so that they never have an unsafe trajectory
  - i.e., need to guarantee *mutually exclusive access* to critical regions
- Classic solution:
  - Semaphores (Edsger Dijkstra)
  - Mutex and condition variables (Pthreads)
  - Monitors (Java)